

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА
ЭКОНОМИКИ»

Школа информатики, физики и технологий

Иванов Максим Сергеевич

Реализация алгоритмов поиска частотных эпизодов для проекта Desbordante

Выпускная квалификационная работа - БАКАЛАВРСКАЯ РАБОТА
по направлению подготовки 01.03.02 Прикладная математика и информатика
образовательная программа «Прикладная математика и информатика»

Руководитель
кандидат физико-математических наук, доцент,
департамент информатики

М. А. Антипов

Соруководитель
кандидат физико-математических наук, ассистент
кафедры информационно-аналитических систем
Математико-механического факультета СПбГУ

Г. А. Чернышев

Санкт-Петербург
2026

Аннотация

Эта работа посвящена реализации алгоритмов поиска частотных эпизодов в системе профилирования данных Desbordante.

Цель работы — реализовать алгоритмы MaxFEM, TKE и AFEM в системе Desbordante, обеспечив существенное ускорение по сравнению с реализациями SPMF и поддержку параллельного выполнения.

В рамках работы в Desbordante добавлен слой представления событийных последовательностей и реализованы три алгоритма поиска частотных эпизодов: MaxFEM (максимальные эпизоды), TKE (k наиболее частотных эпизодов) и AFEM (классическая постановка). Алгоритмы перенесены с Java на C++ с применением системных оптимизаций, для каждого построена параллельная версия. Реализации интегрированы в ядро Desbordante и доступны через нативный Python-интерфейс. Практическая применимость продемонстрирована на задаче обнаружения вторжений по трассам системных вызовов. Часть результатов опубликована в материалах 39-й конференции FRUCT.

Эксперименты на 12 наборах данных в сравнении с реализациями SPMF показали среднее ускорение при пяти потоках — $9,62\times$ (MaxFEM), $11,22\times$ (AFEM) и $24,11\times$ (TKE) — при экономии оперативной памяти в 5–8 раз.

Работа содержит 44 страницы, 4 рисунка, 6 таблиц, использовано 28 источников.

Ключевые слова: поиск частотных эпизодов, событийные последовательности, параллельные алгоритмы, профилирование данных.

Abstract

This work is devoted to implementing Frequent Episode Mining (FEM) algorithms in the Desbordante data-profiling system.

The goal is to implement the MaxFEM, TKE and AFEM algorithms in Desbordante, achieving substantial speedup over SPMF and supporting parallel execution.

A general event-sequence representation layer is introduced into the system, and three FEM algorithms are implemented: MaxFEM (maximal episodes), TKE (k most frequent episodes) and AFEM (classical FEM). The algorithms are ported from Java to C++ with system-level optimisations, and a parallel version is built for each of them. The implementations are integrated into the Desbordante core and exposed through the system's native Python interface. Practical applicability is demonstrated on a host-based intrusion detection task using the ADFA-LD dataset. Part of the results has been published in the proceedings of the 39th IEEE FRUCT conference.

Experimental evaluation on twelve public datasets against the SPMF reference implementations shows mean speedups of $9.62\times$ (MaxFEM), $11.22\times$ (AFEM) and $24.11\times$ (TKE) with five threads, with memory savings of $5\text{--}8\times$.

The thesis contains 44 pages, 4 figures, 6 tables, and 28 references.

Keywords: frequent episode mining, event sequences, parallel algorithms, data mining.

Оглавление

Аннотация	2
Abstract	3
Введение	5
1. Обзор предметной области	7
1.1. Базовые понятия	7
1.2. Меры частотности	8
1.3. Алгоритмы поиска частотных эпизодов	10
1.4. Распирения задачи поиска частотных эпизодов	11
1.5. Выбор алгоритмов для реализации	12
1.6. Фреймворк Desbordante	13
2. Реализация	15
2.1. Описание алгоритмов	15
2.1.1. Алгоритм MaxFEM	16
2.1.2. Алгоритм AFEM	17
2.1.3. Алгоритм ТКЕ	17
2.1.4. Замечание о полноте	20
2.2. Реализация MaxFEM	20
2.3. Реализация AFEM	22
2.4. Реализация ТКЕ	22
2.5. Python-привязки	23
3. Экспериментальная оценка	25
3.1. Методология	25
3.2. Эксперименты	26
3.3. Обсуждение результатов	27
4. Практическое применение	33
4.1. Набор данных ADFA-LD	33
4.2. Признаковое пространство на основе частотных эпизодов	34
4.3. Результаты и сравнение с n -граммной моделью	35
Заключение	39
Список литературы	41

Введение

Профилирование данных [1] занимается извлечением различных типов информации из данных — от простых статистических характеристик до сложных фактов о наличии шаблонов. Такие факты представляют самостоятельную ценность и находят применение в очистке данных, дедупликации, обнаружении аномалий и других задачах. Профилирование может проводиться на данных различной природы: табличных, транзакционных, графовых, событийных. В настоящей работе рассматривается последний тип.

Современные информационные системы генерируют большие объёмы данных в виде последовательностей событий: логи серверов, финансовые транзакции, действия пользователей. Анализ таких данных позволяет обнаруживать скрытые закономерности поведения систем, выявлять аномалии и строить предсказательные модели. Одной из ключевых задач здесь является *поиск частотных эпизодов* (Frequent Episode Mining, FEM) — нахождение подпоследовательностей событий, встречающихся в данных достаточно часто [2]. Рост объёмов данных делает востребованными производительные и масштабируемые решения, способные работать в реальных прикладных задачах.

Стандартная постановка FEM нередко возвращает слишком большое число шаблонов, среди которых много избыточных. Это обусловило появление более специализированных вариантов задачи. Алгоритм MaxFEM [3] реализует поиск *максимальных* частотных эпизодов, исключая из результата шаблоны, содержащиеся в других более длинных эпизодах. Алгоритм TKE [4] предлагает *top- k* формулировку: вместо порога поддержки пользователь задаёт желаемое число результирующих шаблонов k . Алгоритм AFEM [3] решает исходную задачу FEM в классической постановке — без ограничений на вид и число выходных шаблонов.

Эталонные реализации всех трёх алгоритмов входят в библиотеку SPMF [5], однако она не обеспечивает необходимой производительности на больших объёмах данных. Реализации написаны на Java и не используют параллелизм, что существенно ограничивает их применимость при снижении порога поддержки или увеличении длины временного окна. Кроме того, SPMF не интегрирована в экосистему профилирования данных и предназначена преимущественно для демонстрации алгоритмов, а не для использования в реальных пайплайнах.

Desbordante [6] — наукоёмкий высокопроизводительный профилировщик данных с открытым исходным кодом, реализованный на C++ и предоставляющий нативный Python-интерфейс. На момент начала работы в Desbordante отсутствовала поддержка алгоритмов для работы с событийными последовательностями, что делает их реализацию в рамках системы естественным и актуальным направлением развития.

Цель работы — реализовать алгоритмы поиска частотных эпизодов (AFEM,

MaxFEM, TKE) в системе Desbordante, обеспечив существенное ускорение по сравнению с реализациями SPMF и поддержку параллельного выполнения.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести обзор алгоритмов AFEM, MaxFEM и TKE и существующих подходов к их реализации.
2. Реализовать в проекте Desbordante инфраструктуру для работы с событийными последовательностями.
3. Реализовать алгоритмы на C++ с применением системных оптимизаций и интегрировать их в Desbordante.
4. Разработать параллельные версии алгоритмов.
5. Провести экспериментальное сравнение реализованных алгоритмов с реализациями SPMF на реальных наборах данных.
6. Продемонстрировать практическое применение на примере задачи обнаружения вторжений в компьютерные системы.

Структура работы. В главе 1 даётся обзор предметной области: постановка задачи FEM, классификация алгоритмов и описание фреймворка Desbordante. Глава 2 посвящена реализации: описываются принятые решения при переносе алгоритмов на C++ и подход к параллелизации. В главе 3 представлены результаты экспериментального сравнения с реализациями SPMF на 12 наборах данных. Глава 4 содержит демонстрацию применения разработанных инструментов к задаче обнаружения вторжений на основе набора данных ADFA-LD [7]. В заключении подводятся итоги работы и намечаются направления дальнейших исследований.

1. Обзор предметной области

Данная глава формирует теоретическую основу для понимания последующего материала. В ней последовательно вводится понятийный аппарат задачи поиска частотных эпизодов — от базовых определений до различных постановок задачи и обзора алгоритмического ландшафта. В качестве основной опоры здесь используется обзор Ouarem и др. [8]. В конце главы обосновывается выбор конкретных алгоритмов и описывается платформа Desbordante, в которую они интегрированы.

1.1. Базовые понятия

Область поиска частотных эпизодов занимается анализом *событийных последовательностей* — данных, где каждое наблюдение привязано к моменту времени. Такие данные повсеместно встречаются на практике: журналы серверных событий, последовательности биржевых сделок, записи активности пользователей, показания датчиков. В отличие от задач поиска ассоциативных правил, где паттерны ищутся в наборе независимых транзакций, анализ событийных последовательностей направлен на поиск закономерностей в единственной длинной последовательности [2].

Событийные последовательности. Пусть $E = \{i_1, i_2, \dots, i_m\}$ — конечное множество *типов событий*. Подмножество $X \subseteq E$ называется *набором событий* (event set).

Определение 1.1 (Сложная событийная последовательность). Сложная событийная последовательность (*complex event sequence*) — это конечный упорядоченный список

$$S = \langle (SE_{t_1}, t_1), (SE_{t_2}, t_2), \dots, (SE_{t_n}, t_n) \rangle,$$

где $SE_{t_i} \subseteq E$ — набор событий в момент t_i , а метки времени строго возрастают: $t_i < t_j$ при $i < j$.

Пример такой последовательности показан на рисунке 1: восемь наборов событий с временными метками от 1 до 11, не обязанными быть равноотстоящими.

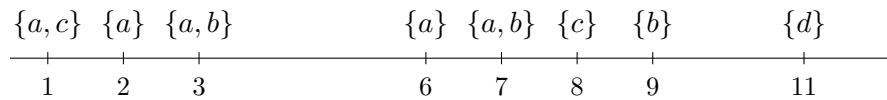


Рис. 1: Пример сложной событийной последовательности

Если в каждый момент происходит ровно одно событие ($|SE_{t_i}| = 1$ для всех i), последовательность называется *простой*. Допустимость одновременных событий — важное обобщение: в реальных системах несколько событий нередко регистрируются с одинаковой меткой времени.

Эпизоды.

Определение 1.2 (Составной эпизод). Составной эпизод (*composite episode*) $\alpha = \langle X_1, X_2, \dots, X_p \rangle$ — это упорядоченный список наборов событий $X_i \subseteq E$, где X_i предшествует X_j при $i < j$. Размер эпизода: $\text{size}(\alpha) = \sum_{i=1}^p |X_i|$.

Различают два частных случая [8]:

- *Последовательный эпизод* (serial episode) — каждый набор содержит одно событие ($|X_i| = 1$); записывается $A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_p$.
- *Параллельный эпизод* (parallel episode) — все события расположены в одном наборе ($p = 1$); порядок между ними не задан.

Составные эпизоды объединяют оба случая: блоки параллельных событий, упорядоченные во времени. Например, $\langle \{A, B\}, \{C\} \rangle$ означает, что A и B происходят одновременно, а затем — C . Задачи, реализованные в данной работе, работают именно с составными эпизодами.

Подэпизоды. Понятие включения эпизода служит основой для определения максимальных эпизодов и ряда свойств мер частотности.

Определение 1.3 (Строгое включение). Эпизод $\alpha = \langle Y_1, \dots, Y_q \rangle$ строго включён в эпизод $\beta = \langle X_1, \dots, X_p \rangle$ (обозначается $\alpha \sqsubseteq \beta$), если существуют индексы $1 \leq k_1 < \dots < k_q \leq p$ такие, что $Y_j \subseteq X_{k_j}$ для всех j . Эпизод α называется подэпизодом β , β — супер-эпизодом α .

Вхождения и окно.

Определение 1.4 (Вхождение). Вхождением эпизода $\alpha = \langle X_1, \dots, X_p \rangle$ в последовательность S называется временной интервал $[t_s, t_e]$ такой, что $X_j \subseteq SE_{z_j}$ для некоторых $t_s = z_1 < z_2 < \dots < z_p = t_e$.

Длина вхождения $t_e - t_s$ характеризует временной «разброс» событий. Пользователь задаёт параметр *winlen* — максимально допустимую длину вхождения — чтобы ограничить поиск «компактными» паттернами. Множество всех вхождений эпизода α с длиной не более *winlen* обозначается $\text{occSet}(\alpha)$.

1.2. Меры частотности

Частота (поддержка, support) эпизода — количественная оценка его регулярности в последовательности. Выбор конкретного способа подсчёта вхождений влияет и на содержательный смысл результатов, и на алгоритмические свойства задачи. Ключевое среди этих свойств — *антимонотонность*: мера f антимонотонна, если для любых эпизодов $\alpha \sqsubseteq \beta$ выполняется $f(\alpha) \geq f(\beta)$. Антимонотонность позволяет безопасно отбрасывать нечастотные кандидаты вместе со всеми их расширениями, принципиально

Таблица 1: Основные меры частотности эпизодов

Мера	Антимонотонна	Представители
Оконная	Да	WINEPI [2], EpiBF [9]
Минимальных вхождений	Нет	MINEPI [2], MEELO [10]
Головная	Нет	EMMA [11], MaxFEM [3], TKE [4]
Неперекрывающихся вхождений	Да	NONEPI [12], POERM [13]
Полная	Да	FEM-DFS [14]

сокращая пространство поиска. В литературе предложено несколько мер [8]; основные из них приведены в таблице 1.

Оконная мера (window-based). Поддержка эпизода равна числу скользящих окон фиксированной ширины, содержащих хотя бы одно его вхождение [2]. Мера антимонотонна, что позволяет эффективно строить уровневые алгоритмы (BFS). Главный недостаток — двойной подсчёт: одно физическое вхождение может попасть сразу в несколько перекрывающихся окон, завышая оценку частотности. Кроме того, ширина окна является дополнительным гиперпараметром, требующим подбора: слишком узкое окно пропускает «растянутые» паттерны, слишком широкое — порождает избыточные совпадения.

Мера минимальных вхождений (minimal occurrence-based). Вхождение $[t_s, t_e]$ называется *минимальным*, если нельзя сдвинуть ни один его конец, не потеряв вхождение [2]. По сравнению с оконной мерой двойной подсчёт устранён, а зависимость от ширины окна снята — каждое «тесное» вхождение учитывается ровно один раз. Ценой этого является утрата антимонотонности, что затрудняет применение уровневых схем отсечения.

Мера неперекрывающихся вхождений (non-overlapped occurrence-based frequency). Подсчитываются только вхождения, не пересекающиеся по времени: если вхождение $[t_s, t_e]$ уже учтено, следующее вхождение должно начинаться не ранее $t_e + 1$. Мера антимонотонна и не страдает двойным подсчётом. Она применяется в алгоритмах NONEPI [12] и POERM [13], ориентированных на поиск эпизодных правил, и особенно удобна при анализе плотных последовательностей, где перекрывающихся вхождений много.

Полная мера (total frequency). Считается суммарное количество вхождений всех возможных типов (с учётом перекрытий и временных сдвигов). Мера антимонотонна; она реализована в алгоритме FEM-DFS [14] как часть гибкой схемы, поддерживающей несколько определений частотности.

Головная мера (head frequency). В настоящей работе используется *головная мера частотности* (head frequency), введённая в [11].

Определение 1.5 (Головная мера частотности). Поддержка эпизода α в последова-

тельности S — число различных начальных моментов среди всех его вхождений:

$$\text{sup}(\alpha) = |\{t_s \mid [t_s, t_e] \in \text{occSet}(\alpha)\}|.$$

Смысл меры интуитивен: считается, сколько раз паттерн «запускается» в разные моменты времени, независимо от числа способов его завершения. Это делает головную меру особенно пригодной для задач предсказания: нас интересует частота *начала* паттерна, а не число всех его развёрток.

Головная мера *не является антимонотонной* в общем случае: поддержка эпизода может превышать поддержку произвольного подэпизода. Тем не менее для *префиксов* антимонотонность сохраняется: если $\alpha = \langle X_1, \dots, X_q \rangle$ является префиксом $\beta = \langle X_1, \dots, X_q, \dots, X_p \rangle$, то каждое вхождение β начинается в момент, являющийся также начальным моментом некоторого вхождения α , откуда $\text{sup}(\alpha) \geq \text{sup}(\beta)$. Это свойство используется в алгоритмах семейства ЕММА [11] для отсеечения: если текущий эпизод нечастотен, его последовательные расширения заведомо тоже нечастотны.

1.3. Алгоритмы поиска частотных эпизодов

Центральная алгоритмическая задача в FEM — эффективная навигация по пространству кандидатов, размер которого экспоненциален по числу типов событий. Прямой перебор неприемлем даже для умеренных входных данных: ключом к практической применимости служит *отсечение* — исключение заведомо нечастотных кандидатов без явной проверки. Возможности отсеечения напрямую определяются алгоритмической стратегией обхода и монотонностью выбранной меры частотности [8].

Обход в ширину (BFS). Стратегия обхода в ширину (уровневый поиск) подразумевает итеративную генерацию паттернов нарастающей длины. На основе уже найденных частотных k -эпизодов синтезируются кандидаты размера $(k + 1)$. Затем для каждого из них вычисляется поддержка, и элементы, не преодолевшие заданный порог, отсекаются. Если мера антимонотонна, отсечённый кандидат тянет за собой всё своё поддерево. Классическим примером здесь служит WINEPI [2], где поиск вхождений реализован через конечные автоматы внутри скользящего окна. Позже этот подход был обобщен в алгоритме EpiBF [9] для паттернов без жесткого ограничения по длине. Главное ограничение семейства BFS — высокое потребление памяти. Алгоритм вынужден хранить весь срез кандидатов текущего уровня одновременно, что в условиях объемного словаря событий быстро приводит к исчерпанию ресурсов.

Обход в глубину (DFS). Алгоритмы, использующие эту стратегию, рекурсивно наращивают один эпизод, сразу проверяя его частотность; при нарушении условия — откатываются. В памяти в каждый момент хранится только текущая ветка дерева поиска, а не весь уровень — что принципиально снижает потребление памяти.

FEM-DFS [14] реализует гибкий DFS с поддержкой нескольких мер частотности для простых последовательностей; MANEPI [15] добавляет поддержку временных ограничений на расстояние между событиями.

Выбор стратегии обхода тесно связан с выбором меры частотности. BFS естественно сочетается с антимонотонными мерами: именно гарантия монотонности делает уровневое отсечение корректным. DFS более гибок: он способен использовать более слабые монотонные свойства там, где полная антимонотонность недостижима, — в частности, свойство префикса для головной меры, рассмотренное в предыдущем разделе. Это взаимодействие стратегии и меры определяет архитектуру алгоритмов, реализованных в данной работе, и детально рассматривается в главе 2.

1.4. Расширения задачи поиска частотных эпизодов

Стандартная постановка задачи FEM — поиск всех частотных эпизодов при заданных пороге $minsup$ и длине окна $winlen$ — обладает рядом практических ограничений, что побудило исследователей к разработке расширений [8].

Краткие представления. При малых значениях $minsup$ алгоритм может вернуть миллионы эпизодов, значительная часть которых является избыточной: если эпизод β частотен, то, как правило, частотны и все его подэпизоды $\alpha \sqsubseteq \beta$. Для устранения избыточности предложены краткие представления.

Замкнутые эпизоды (closed episodes) — частотные эпизоды, у которых нет супер-эпизода с той же поддержкой [8]. По множеству замкнутых эпизодов однозначно восстанавливается полный набор частотных эпизодов вместе с их поддержками: подэпизод α , не входящий в это множество, наследует поддержку любого ближайшего замкнутого надэпизода. Тем самым повторное сканирование последовательности не требуется. Алгоритмы FCEMiner [16], CloEpi [17], MineEpisode [18] реализуют эту идею для различных мер частотности.

Максимальные эпизоды [3] (maximal episodes):

Определение 1.6 (Максимальный эпизод). *Частотный эпизод α называется максимальным, если не существует частотного эпизода β такого, что $\alpha \sqsubseteq \beta$.*

Множество максимальных эпизодов является подмножеством замкнутых и ещё более компактно. Как показывают эксперименты, переход к максимальным эпизодам позволяет сократить итоговую выдачу в 2,5–4 раза по сравнению с полным множеством [3]. Для нахождения таких паттернов в составных последовательностях применяется специализированный алгоритм MaxFEM [3].

Поиск top- k эпизодов (top- k episode mining). Подбор подходящего $minsup$ без предварительных знаний о данных трудоёмок. Незначительное снижение порога может увеличить число результатов на несколько порядков, а завышенный порог про-

пускает значимые паттерны. Постановка задачи top- k предлагает альтернативу: пользователь просто указывает желаемое количество результатов k . Алгоритм **ТКЕ** [4] реализует эту идею для составных эпизодов с головной мерой частотности.

Поиск с ограничениями (constraint-based episode mining). Пользователи часто хотят ограничить паттерны дополнительными условиями: например, максимальным временным расстоянием между событиями (span) или допустимым зазором между двумя последовательными событиями (gap). Алгоритмы DiscoveryTotal [19], WinMiner [20] и PPT [21] интегрируют такие ограничения непосредственно в поиск, что позволяет быстрее отбрасывать кандидатов и получать более специфичные результаты.

Эпизодные правила (episode rules). Правило вида $\alpha \rightarrow \beta$ означает, что эпизод α систематически предшествует β в последовательности [8]. Такие правила находят применение в задачах предсказания и обнаружения аномалий: они позволяют формализовать причинно-следственные связи между паттернами и отвечать на вопрос «если произошло α , что произойдёт далее?». Поиск эпизодных правил был предложен ещё в [2] и с тех пор активно развивается; среди алгоритмов выделяются WinMiner [20], NONEPI [12] и POERM [13].

Онлайн-обработка потоков (online frequent episode mining). В ряде приложений событийная последовательность непрерывно пополняется, и повторный запуск алгоритма с нуля при каждом обновлении нецелесообразен. Онлайн-алгоритмы (Extractor [22], MESELO [23], ONCE/ONCE+ [24]) инкрементально обновляют найденные паттерны по мере поступления новых событий, обрабатывая только «актуальное» окно последовательности.

Высокоутилитарный поиск (high utility episode mining). Стандартная задача FEM полагает все события равнозначными. В высокоутилитарной формулировке каждому типу события приписана «полезность» (utility) — например, прибыль от финансовой операции — и цель состоит в нахождении эпизодов с суммарной полезностью выше порога [8]. Задача принципиально сложнее FEM, поскольку функция полезности не антимонотонна.

1.5. Выбор алгоритмов для реализации

Из рассмотренного многообразия подходов в данной работе реализованы три алгоритма: AFEM, MaxFEM и ТКЕ — все из семейства ЕММА [11], работающие с составными эпизодами и головной мерой частотности. Выбор обусловлен следующими соображениями.

Охват основных постановок. Три алгоритма вместе охватывают три принципиально разных способа формулировки задачи пользователем: через абсолютный порог

(AFEM), через требование максимальности (MaxFEM) и через желаемое число результатов (TKE). Пользователь может выбрать наиболее удобный вариант в зависимости от задачи.

Практическая ценность каждого алгоритма. AFEM даёт полное множество паттернов, необходимое для задач, где важны все частотные закономерности без исключения. MaxFEM возвращает компактное представление — удобное там, где обилие избыточных подпаттернов затрудняет анализ. TKE особенно ценен при исследовательском анализе, когда подходящий порог заранее неизвестен.

Общая основа. Все три алгоритма наследуют поисковую процедуру ЕММА; AFEM и MaxFEM отличаются лишь стратегией фильтрации результатов, TKE — механизмом обновления порога. Это означает, что реализация общей инфраструктуры (парсинг данных, структуры вхождений, параллелизация) приносит выгоду сразу трём алгоритмам.

Эталонная реализация для сравнения. Все три алгоритма реализованы в библиотеке SPMF [5] — широко используемом Java-инструментарии для профилирования данных. Это даёт возможность верифицировать корректность реализации, а также провести сравнительный анализ производительности на тех же наборах данных, на которых тестировались оригинальные версии.

1.6. Фреймворк Desbordante

Desbordante [6] — высокопроизводительный профилировщик данных с открытым исходным кодом¹. Система реализована на C++ и предоставляет нативный Python-интерфейс через пакет, публикуемый на PyPI.

Архитектура Desbordante основана на принципе *алгоритма-плагины*: каждый алгоритм — отдельный класс, реализующий унифицированный интерфейс (загрузка данных, задание параметров, запуск, получение результатов). За счет такой структуры добавлять новые методы можно без какого-либо вмешательства в ядро системы. Кроме того, для конечного пользователя процесс запуска алгоритмов выглядит единообразно вне зависимости от конкретной задачи.

До начала данной работы Desbordante поддерживал профилирование табличных и транзакционных данных: поиск функциональных зависимостей, ассоциативных правил, зависимостей включения и других примитивов. Событийные последовательности как тип данных в системе отсутствовали полностью. Реализация алгоритмов FEM потребовала разработки соответствующей инфраструктуры с нуля: формат входных данных, Python-обёртки для последовательностей и наборов событий, интеграция результатов в общую схему получения паттернов.

¹<https://github.com/Desbordante/desbordante-core>

Выбор Desbordante в качестве платформы обусловлен тремя факторами. Во-первых, C++-ядро системы обеспечивает управление памятью и производительность, недостижимые в JVM-языках. Во-вторых, нативный Python-интерфейс позволяет встраивать алгоритмы в аналитические пайплайны без промежуточных файлов и конвертации форматов — в отличие от SPMF [5], где интеграция с Python требует запуска подпроцесса JVM и разбора текстового вывода.

Итог главы. Введён формальный аппарат задачи поиска частотных эпизодов: сложные событийные последовательности, составные эпизоды, вхождения и головная мера частотности. Дан обзор алгоритмического ландшафта — от уровневых алгоритмов WINEPI/MINEPI до семейства ЕММА, к которому относятся реализованные алгоритмы. Рассмотрены ключевые расширения стандартной задачи, обоснован выбор алгоритмов AFEM, MaxFEM и ТКЕ. Следующая глава посвящена деталям их реализации в системе Desbordante.

2. Реализация

В данной главе рассматриваются алгоритмы поиска частотных эпизодов, реализованные в рамках настоящей работы, их детали на уровне реализации на языке C++, применённые оптимизации, а также интеграция в систему Desbordante. Сначала описываются оригинальные алгоритмы в том виде, в каком они представлены в исходных публикациях, после чего обсуждаются особенности их реализации.

2.1. Описание алгоритмов

Все три реализованных алгоритма — MaxFEM, AFEM и TKE — опираются на общую алгоритмическую основу: подход ЕММА [11], в котором для обхода пространства поиска применяется поиск в глубину, для подсчёта поддержки — головная мера частотности, а для хранения вхождений — две ключевые структуры данных. Первая из них, *список позиций* (location list), хранит временные метки вхождений параллельного эпизода. Вторая, *список границ* (bound list), хранит интервалы вхождений составных эпизодов.

Определение 2.1 (Список позиций). Список позиций события e в последовательности S — упорядоченный список временных меток, в которых e встречается:

$$locList(e) = \{ t_i \mid e \in SE_{t_i} \}.$$

Поддержка события равна мощности этого списка: $\sup(e) = |locList(e)|$.

Список позиций параллельного эпизода $ep \cup \{e\}$ вычисляется пересечением списков его компонент:

$$locList(ep \cup \{e\}) = locList(ep) \cap locList(e).$$

Определение 2.2 (Список границ). Список границ параллельного эпизода pe в перекодированной последовательности S' — множество точечных интервалов:

$$boundList(pe) = \{ [t, t] \mid pe \subseteq SE_t \in S' \}.$$

Список границ составного эпизода $\langle ep, pe \rangle$, полученного серийным расширением ep эпизодом pe :

$$boundList(\langle ep, pe \rangle) = \{ [u, w] \mid [u, v] \in boundList(ep), [w, w] \in boundList(pe), \\ w - u \leq winlen, v < w \}.$$

Поддержка составного эпизода ep равна числу уникальных левых концов его границ:

$$\sup(ep) = \left| \{ t_s \mid [t_s, t_e] \in boundList(ep) \} \right|.$$

2.1.1. Алгоритм MaxFEM

MaxFEM [3] решает задачу поиска максимальных частотных эпизодов: из всех частотных эпизодов выбираются лишь те, которые не входят ни в какой другой частотный эпизод. Это позволяет существенно сократить число возвращаемых паттернов, сохраняя при этом полноту представления.

Входными параметрами алгоритма служат последовательность событий S , порог минимальной поддержки $minsup$ и максимальная длина окна $winlen$. Работа алгоритма разделена на следующие шаги.

Шаг 1. Нахождение частотных событий. Алгоритм однократно сканирует входную последовательность и подсчитывает поддержку каждого отдельного события — число временных меток, в которых оно встречается. События с поддержкой ниже $minsup$ исключаются из дальнейшего рассмотрения.

Шаг 2. Построение списков позиций. Алгоритм выполняет повторный проход по последовательности для обработки оставшихся частотных событий. Для каждого из них формируется список позиций — упорядоченный массив временных меток, фиксирующий все вхождения конкретного события. Поддержка события равна числу элементов в этом списке.

Шаг 3. Нахождение частотных параллельных эпизодов. Алгоритм рекурсивно порождает параллельные расширения. Каждый частотный параллельный эпизод ep расширяется добавлением события e . Список позиций нового эпизода вычисляется пересечением списков позиций компонент. Новый эпизод $ep \cup \{e\}$ является частотным, если его поддержка не ниже $minsup$. В итоге все найденные частотные параллельные эпизоды собираются и накапливаются в множестве $PEpisodes$.

Шаг 4. Перекодирование последовательности. Каждому эпизоду из $PEpisodes$ присваивается уникальный числовой идентификатор. Исходная последовательность S преобразуется в перекодированную S' : каждое событие заменяется идентификатором того параллельного эпизода, который это событие представляет. Такое перекодирование позволяет единообразно строить составные эпизоды на следующем шаге.

Шаг 5. Нахождение максимальных составных эпизодов. Алгоритм инициализирует рабочее множество W , добавляя в него все элементы $PEpisodes$ в качестве кандидатов в максимальные. Далее производятся *серийные расширения*: к составному эпизоду ep дописывается параллельный эпизод pe , образуя более длинный составной эпизод $\langle ep, pe \rangle$. Поддержка серийного расширения вычисляется через список границ по введённым определениям.

Когда найден новый частотный составной эпизод X , он сравнивается с уже имеющимися в W . Если X строго включён в некоторый эпизод из W , то X немаксимален и отбрасывается. В противном случае X добавляется в W , а все элементы W , строго

включённые в X , удаляются. Инвариант гарантирует, что по завершении алгоритма W содержит именно максимальные частотные эпизоды.

Оптимизации. MaxFEM применяет три стратегии для сокращения времени работы [3].

1. *EFE (Efficient Filtering of Non-maximal Episodes)*. Структура W реализована как массив куч, индексированных по размеру хранимых эпизодов. При проверке, является ли эпизод размера s максимальным, достаточно просматривать лишь кучи размеров больше s (проверка надключения) или меньше s (проверка подключения). Дополнительно вводится хеш-функция на эпизодах — сумма числовых кодов их событий — которая позволяет досрочно прерывать поиск по куче, не доходя до всех её элементов.
2. *SEC (Skip Extension Checking)*. Если эпизод ep породил хотя бы одно частотное серийное расширение, он не является максимальным и дополнительная проверка его максимальнойности не нужна. Это позволяет пропускать дорогостоящие сравнения с W для таких эпизодов.
3. *TP (Temporal Pruning)*. При построении списка границ алгоритм останавливает вычисление досрочно, как только становится ясно, что оставшиеся элементы обоих списков не могут обеспечить $minsup$ вхождений. Это позволяет избежать полного обхода списков для заведомо нечастотных расширений.

Псевдокод алгоритма приведён в виде Алгоритма 1.

2.1.2. Алгоритм AFEM

AFEM (All Frequent Episode Miner) решает классическую задачу FEM — поиск *всех* частотных эпизодов в последовательности. По структуре он практически идентичен MaxFEM: используются те же шаги построения списков позиций, нахождения параллельных эпизодов, перекодирования последовательности и серийного расширения составных эпизодов. Принципиальное отличие состоит в отсутствии механизма фильтрации немаксимальных паттернов: каждый найденный частотный эпизод включается в итоговый результат. Алгоритм был реализован как дополнительный в рамках той же кодовой базы, что и MaxFEM, с минимальными изменениями в логике поиска.

2.1.3. Алгоритм TKE

TKE (Top-K Episode Mining) [4] переопределяет постановку задачи: вместо фиксированного порога $minsup$ пользователь задаёт число k , и алгоритм возвращает k наиболее частотных эпизодов. Такой подход устраняет необходимость подбирать пороговое значение, которое на практике зависит от неизвестных характеристик данных.

Algorithm 1: MaxFEM [3]

Input: S — входная последовательность; $minsup$ — минимальная поддержка; $winlen$ — длина окна

Output: множество максимальных частотных эпизодов

- 1 Просмотреть S , вычислить $\sup(e)$ для каждого события $e \in E$;
- 2 $E' \leftarrow \{e \mid e \in E, \sup(e) \geq minsup\}$;
- 3 Удалить из S все $e \notin E'$;
- 4 Построить $locList(e)$ для каждого $e \in E'$;
- 5 $PEpisodes \leftarrow E'$;
- 6 **foreach** параллельный эпизод $ep \in PEpisodes$ **do**
- 7 **foreach** $e \in E'$ такое, что $e \notin ep$ и $\forall f \in ep: f \prec e$ **do**
- 8 $newE \leftarrow parallelExtension(ep, e)$;
- 9 **if** $\sup(newE) \geq minsup$ **then**
- 10 $PEpisodes \leftarrow PEpisodes \cup \{newE\}$;
- 11 $W \leftarrow PEpisodes$;
- 12 Перекодировать S в S' через идентификаторы параллельных эпизодов;
- 13 **foreach** составной эпизод $ep \in W$ **do**
- 14 **foreach** $pe \in PEpisodes$ **do**
- 15 $newE \leftarrow serialExtension(ep, pe)$;
- 16 **if** $\sup(newE) \geq minsup$ и W не содержит надэпизода $newE$ **then**
- 17 $W \leftarrow W \cup \{newE\}$;
- 18 Удалить из W все подэпизоды $newE$;
- 19 **return** W ;

Алгоритм использует тот же механизм поиска, что ЕММА и MaxFEM, но порог $minsup$ не задаётся заранее: ТКЕ стартует с $minsup = 1$ и повышает его по мере нахождения новых эпизодов.

Шаг 1. Нахождение top- k событий. Алгоритм сканирует последовательность и подсчитывает поддержку каждого события. Затем $minsup$ устанавливается равным поддержке k -го по убыванию события — это оптимизация SEI (Single Episode Increase), позволяющая сразу повысить порог и сократить множество кандидатов. Для оставшихся частотных событий строятся списки позиций.

Шаг 2. Нахождение top- k параллельных эпизодов. Аналогично MaxFEM производятся рекурсивные параллельные расширения. Отличие состоит в том, что после добавления нового частотного параллельного эпизода в $PEpisodes$ порог $minsup$ немедленно обновляется: он устанавливается равным поддержке k -го элемента в $PEpisodes$, а все эпизоды ниже нового порога удаляются. Это позволяет как можно раньше отсеять нерелевантных кандидатов.

Шаг 3. Перекодирование последовательности. Выполняется так же, как в MaxFEM: top- k параллельным эпизодам присваиваются идентификаторы, исходная

Algorithm 2: TKE [4]

Input: S — входная последовательность; k — число искомых эпизодов; $winlen$ — длина окна

Output: top- k частотных эпизодов

```
1  $minsup \leftarrow 1$ ;  
2 Просмотреть  $S$ , вычислить  $\sup(e)$  для каждого  $e \in E$ ;  
3  $minsup \leftarrow \sup$   $k$ -го наиболее частотного события в  $E$  ; // SEI  
4  $E' \leftarrow \{e \mid e \in E, \sup(e) \geq minsup\}$ ;  
5 Удалить из  $S$  все  $e \notin E'$ ;  
6 Построить  $locList(e)$  для каждого  $e \in E'$ ;  
7  $PEpisodes \leftarrow E'$ ;  
8 foreach параллельный эпизод  $ep \in PEpisodes$  do  
9   foreach  $e \in E'$  такое, что  $e \notin ep$  и  $\forall f \in ep: f \prec e$  do  
10      $newE \leftarrow parallelExtension(ep, e)$ ;  
11     if  $\sup(newE) \geq minsup$  then  
12        $PEpisodes \leftarrow PEpisodes \cup \{newE\}$ ;  
13        $minsup \leftarrow \sup$   $k$ -го элемента  $PEpisodes$ ;  
14 Перекодировать  $S$  в  $S'$  через идентификаторы параллельных эпизодов;  
15  $CEpisodes \leftarrow PEpisodes$ ;  
16 foreach составной эпизод  $ep \in CEpisodes$  do  
17   foreach  $pe \in PEpisodes$  do  
18      $newE \leftarrow serialExtension(ep, pe)$ ;  
19     if  $\sup(newE) \geq minsup$  then  
20        $CEpisodes \leftarrow CEpisodes \cup \{newE\}$ ;  
21        $minsup \leftarrow \sup$   $k$ -го элемента  $CEpisodes$ ;  
22 return  $CEpisodes$ ;
```

последовательность перекодируется.

Шаг 4. Нахождение top- k составных эпизодов. Формируется множество $CEpisodes$, изначально совпадающее с $PEpisodes$. Далее производятся серийные расширения аналогично MaxFEM. После каждого нового частотного составного эпизода $minsup$ повышается до поддержки k -го элемента $CEpisodes$, а эпизоды ниже порога удаляются. По завершении алгоритма $CEpisodes$ содержит top- k эпизодов.

Оптимизация динамического поиска (Dynamic Search). TKE поддерживает приоритетную очередь эпизодов-кандидатов, упорядоченную по убыванию поддержки. На каждом шаге алгоритм расширяет наиболее частотный из доступных кандидатов. Это позволяет быстрее повысить $minsup$ и, следовательно, раньше отсеять большую часть пространства поиска [4].

Псевдокод алгоритма приведён в виде Алгоритма 2.

2.1.4. Замечание о полноте

Свойство антимонотонности поддержки в случае головной меры выполняется лишь для *префиксных* подэпизодов: если $\alpha = \langle X_1, \dots, X_p \rangle$, то $\text{sup}(\alpha) \leq \text{sup}(\langle X_1, \dots, X_i \rangle)$ для любого $i < p$. Для произвольного подэпизода, начинающегося с другого компонента, это соотношение нарушается: поддержка $\langle \{a\}, \{e\} \rangle$ считается по числу различных начальных позиций $\{a\}$ и может многократно превышать $\text{sup}(\{e\})$.

Следствием этого является то, что MaxFEM и AFEM, удаляя в шаге 1 все события с поддержкой ниже *minsup*, могут пропустить часть частотных составных эпизодов. Таким образом, оба алгоритма технически решают несколько более узкую задачу. На практике это ограничение несущественно: пропущенные эпизоды содержат редкие события, что само по себе снижает их практическую ценность.

ТКЕ в этом отношении менее подвержен данной проблеме: алгоритм стартует с *minsup* = 1 и повышает порог лишь по мере нахождения k эпизодов, поэтому нечастотные по отдельности события сохраняются дольше и могут войти в состав частотных составных эпизодов [4]. Однако именно это динамическое повышение порога вносит элемент недетерминированности: итоговое множество top- k эпизодов зависит от порядка обработки кандидатов, и разные реализации при одинаковых входных данных могут возвращать немного различные результаты.

2.2. Реализация MaxFEM

Исходная реализация MaxFEM выполнена на Java в составе библиотеки SPMF [5]. В рамках настоящей работы алгоритм переписан на C++: прямое управление памятью и отсутствие сборщика мусора устраняют непредсказуемые паузы и снижают пиковое потребление памяти.

Повсеместно применяются `reserve()` и семантика перемещения (`std::move`). Когда требуемая ёмкость структуры данных известна заранее — например, ёмкость списка позиций события определяется его поддержкой, подсчитанной на первом шаге — память выделяется однократно, без перевыделений. Передача владения большими объектами (новыми эпизодами, списками позиций и границ) осуществляется через перемещение вместо копирования. Дополнительно, если после построения нового списка границ его ёмкость существенно превышает размер, неиспользуемая память освобождается явным вызовом `shrink_to_fit()`.

Существенную роль играет и компоновка данных в памяти. В Java-реализации список границ представлен как `ArrayList<int[]>`: каждая граница $[u, v]$ выделяется как отдельный массив-объект на куче, доступный по указателю. Список позиций хранится как `List<Integer>` с боксингом каждого числа в объект. В C++ оба списка реализованы как непрерывные векторы структур фиксированного размера — все элементы расположены в памяти смежно. При последовательном обходе, который

является основной операцией на шаге 5, это даёт существенно лучшее использование кэша по сравнению с разрозненными объектами Java.

Ещё одна оптимизация шагов 1–4 состоит в следующем. После отсеивания нечастотных событий оставшиеся перенумеровываются в непрерывный диапазон $[0, N)$. Это позволяет хранить все данные по каждому событию — включая список его позиций — в обычных массивах с доступом по индексу, без косвенности. В Java-реализации та же роль отведена хеш-таблицам, где каждое обращение включает вычисление хеша и разрешение коллизий.

Параллелизация

Шаг 5 — обход дерева серийных расширений — подавляюще доминирует по времени выполнения; шаги 1–4 работают последовательно.

Деревья DFS для разных частотных параллельных эпизодов независимы: результаты одного не влияют на ход обхода другого. Это позволяет каждому параллельному эпизоду сопоставить отдельную задачу — независимый обход DFS, в котором он является первым компонентом строящегося составного эпизода — и распределить задачи между потоками пула Boost.Asio.

Размеры деревьев DFS сильно варьируются: одни параллельные эпизоды порождают единицы серийных расширений, другие — тысячи. Чтобы потоки, завершившие обход небольших деревьев, не простаивали, применяется адаптивное динамическое дробление задач. Когда в ходе обхода поток находит частотное серийное расширение, он проверяет число активных задач в пуле: если оно не превышает заданного порога — текущая ветка выносится в пул как отдельная задача, и свободный поток может немедленно подхватить её; если порог достигнут — ветка обрабатывается последовательно в текущем потоке, чтобы не создавать лишних накладных расходов от бесчисленного количества мелких задач.

В ходе поиска потоки не обмениваются промежуточными данными: каждый накапливает найденные максимальные эпизоды в собственном локальном буфере. Мьютекс используется только для короткой операции передачи заполненного буфера в общий список по завершении задачи — это единственная точка синхронизации. В локальных буферах временно могут присутствовать эпизоды, которые в итоге окажутся не максимальными, — небольшая плата дополнительной памятью за устранение конкуренции при записи в ходе поиска. Сама фильтрация не максимальных эпизодов выполняется уже после завершения всех потоков.

При слиянии буферов реализован приём, исключаящий операции удаления из итоговой коллекции: эпизоды обрабатываются в порядке убывания длины, с хеш-фильтрацией перед полной проверкой включения. В момент рассмотрения очередного кандидата все более длинные эпизоды уже добавлены — достаточно проверить, не

поглощается ли он одним из них. Поглощённые более короткие кандидаты просто не попадают в коллекцию.

2.3. Реализация AFEM

AFEM реализован в той же кодовой базе, что и MaxFEM. Единственное содержательное отличие — отсутствие проверки максимальности: каждый найденный частотный составной эпизод сразу включается в результат.

2.4. Реализация ТКЕ

Исходная реализация ТКЕ выполнена на Java в составе SPMF [5] как однопоточная. В рамках настоящей работы алгоритм переписан на C++ с теми же системными оптимизациями, что и MaxFEM: компактная нумерация событий, непрерывные массивы для списков позиций и границ, повсеместное применение `reserve()` и `std::move`. Помимо этого, для построения списков границ используется тот же механизм раннего прерывания, что и в MaxFEM: если оставшихся элементов заведомо не хватает для достижения *minsup*, вычисление прекращается досрочно. SPMF-реализация ТКЕ использует вариант без раннего прерывания.

Управление текущим множеством *top-k* устроено следующим образом. Параллельно поддерживаются две очереди `std::priority_queue: top-k` — очередь с минимумом наверху, хранящая не более *k* эпизодов, с доступом к текущему порогу *minsup* за $O(1)$, и *explore* — очередь с максимумом наверху, хранящая кандидатов на расширение. Приоритет *explore* по убыванию поддержки реализует оптимизацию Dynamic Search [4]: наиболее перспективные эпизоды обрабатываются первыми и быстрее поднимают порог. При добавлении нового эпизода обе очереди обновляются совместно, гарантируя что *top-k* никогда не превышает *k* эпизодов. В Java-реализации добавление и очистка разделены: после добавления запускается цикл извлечения минимальных элементов, допуская кратковременный выход за границу *k*.

Параллелизация

В отличие от MaxFEM, в ТКЕ параллелизуются оба этапа — поиск параллельных эпизодов и поиск составных эпизодов. Время работы первого этапа сопоставимо со временем второго, что делает его параллелизацию существенной.

Применить к ТКЕ подход MaxFEM — разбить пространство поиска на независимые поддеревья и выдать каждое отдельному потоку — невозможно: порог меняется в ходе поиска, и поддерево, актуальное в начале, может стать нерелевантным к тому моменту, когда поток доберётся до его глубин.

Поэтому применяется схема «производитель — потребитель». Главный поток

управляет обходом пространства поиска и чередует два вида деятельности: либо выбирает следующего кандидата из `explore` и передаёт рабочим потокам задачу — вычислить его расширения — либо вычитывает готовые результаты, обновляет `top-k` и `explore`, и при повышении порога записывает новое значение в атомарную переменную. Рабочие потоки через слабую атомарную нагрузку читают актуальный порог и отбрасывают устаревшие задачи до вычислений. Очереди между главным и рабочими потоками (задания и результаты) реализованы через lock-free структуры `Boost.Lockfree`, исключая блокировки в горячем пути. Рабочие потоки ожидают задач на счётном семафоре.

Главный поток является единственным писателем `top-k` и `explore`. Это исключает гонки данных без дополнительных блокировок, но накладывает ограничение: заполнение очереди заданий ограничено, чтобы главный поток успевал обрабатывать результаты, а не только раздавать задачи. Порог устанавливается кратным числу рабочих потоков.

При поиске `top-k` составных эпизодов применяется аналогичная схема с дополнительной оптимизацией. Параллельные эпизоды отсортированы по убыванию поддержки. При формировании задачи главный поток вычисляет, сколько из них имеют поддержку не ниже актуального *minsup*, и передаёт рабочим потокам только это число. Рабочие потоки расширяют текущий составной эпизод только этим количеством кандидатов, пропуская остальных без вычислений.

2.5. Python-привязки

Python стал стандартом де-факто в области анализа данных, поэтому нативный Python-интерфейс критичен для практического применения инструмента. `Desbordante` поставляется как Python-пакет, распространяемый через PyPI.

Альтернативный путь — вызов SPMF через подпроцесс — сопряжён с рядом практических неудобств. Пользователь вынужден сериализовывать данные в текстовый SPMF-формат, записывать их на диск, запускать Java Virtual Machine и затем разбирать результаты обратно. Это создаёт накладные расходы на ввод-вывод и делает интеграцию в аналитические пайплайны трудоёмкой.

`Desbordante` устраняет эти ограничения: входная последовательность может передаваться непосредственно как Python Iterable, где каждый элемент — набор событий, опционально снабжённый временной меткой. Данные, вычисления и результаты остаются в едином адресном пространстве без промежуточного диска. Для сценариев, где данные уже хранятся в текстовом формате, предусмотрена и загрузка из файла.

Итог главы. В данной главе рассмотрены три реализованных алгоритма — MaxFEM, AFEM и ТКЕ. Для каждого из них переход с Java на C++ сопровождался системными оптимизациями: компактной нумерацией событий, непрерывным размещением

структур данных в памяти и управлением памятью без сборщика мусора. Параллелизация позволяет эффективно задействовать многоядерные процессоры. Все три алгоритма доступны через нативный Python-интерфейс без зависимостей от JVM и промежуточных файлов.

3. Экспериментальная оценка

В данной главе проводится экспериментальная оценка трёх реализованных алгоритмов — MaxFEM, TKE и AFEM — в сравнении с их эталонными реализациями из библиотеки SPMF [5]. Для каждого алгоритма исследуются выигрыш в скорости и экономия памяти, достигаемые как за счёт переноса с Java на C++, так и за счёт параллелизации. Кроме того, отдельно рассматривается масштабируемость параллельных реализаций при увеличении числа потоков.

3.1. Методология

Аппаратная конфигурация. Основные эксперименты проводились на ноутбуке ASUS Vivobook S 14 с процессором AMD Ryzen 5 7535HS (6 физических ядер, 12 аппаратных потоков) и 16 ГБ оперативной памяти под управлением Linux. Для исследования масштабируемости при большом числе потоков использовался отдельный облачный стенд: виртуальная машина с процессором Intel Xeon Gold 6338 (2,0 ГГц, 32 физических ядра, 64 vCPU) и 64 ГБ оперативной памяти.

Изоляция ядер. Для обеспечения воспроизводимости результатов применялся стандартный набор мер по изоляции стенда. Пять из шести физических ядер процессора были исключены из пула задач ОС с помощью утилиты `tuna` и переведены в бестиковый режим (`nohz_full`) через параметры загрузчика; обработка прерываний закреплялась за оставшимся ядром. При этом из каждого физического ядра использовался только один аппаратный поток, что исключает влияние SMT на результаты. Дополнительно отключались турбо-ускорение и подкачка, а регулятор частоты фиксировался в режиме `performance`. Измерения выполнялись в контейнере Docker с жёстким ограничением памяти и явной привязкой к изолированным ядрам.

Программное окружение. Реализации C++ собирались компилятором GCC с флагом оптимизации `-O3` (тип сборки `Release`). Реализации SPMF выполнялись на OpenJDK 21. Время выполнения и пиковое потребление памяти (RSS) измерялись утилитой GNU Time.

Параметры запуска. Длина окна принималась равной $winlen \in \{5, 10, 15\}$. Для каждого датасета и каждого значения $winlen$ порог $minsup$ подбирался так, чтобы время выполнения базовой реализации SPMF находилось в целевом диапазоне от нескольких секунд до нескольких минут, охватывая тем самым разнообразные вычислительные нагрузки.

Датасеты. В экспериментах использовались 12 реальных датасетов из репозитория SPMF, охватывающих различные предметные области: анализ трафика, потоки кликов, розничную торговлю, биологию, сенсорные данные и другие. Характеристики датасетов приведены в таблице 2. По масштабу, средней длине транзакции и плотности датасеты существенно различаются, что позволяет оценить поведение алгоритмов в разнообразных условиях.

Таблица 2: Характеристики датасетов

Датасет	Транз.	Элементы	Ср. длина	Плотн. (%)	Тип
accidents	340 183	468	33,80	7,22	Трафик
BMS1	59 602	497	2,51	0,51	Поток кликов
BMS2	77 512	3 340	4,62	0,14	Поток кликов
chainstore	1 112 949	46 086	7,23	0,02	Продукты
chess	3 196	75	37,00	49,33	Игровые ходы
kosarak	990 002	41 270	8,10	0,02	Поток кликов
mushrooms	8 416	119	23,00	19,33	Биология
OnlineRetail	541 909	2 603	4,37	0,17	Эл. коммерция
PAMAP	1 000 000	141	23,93	16,97	Датчики
RecordLink	574 913	29	10,00	34,48	Связывание
retail	88 162	16 470	10,30	0,06	Ритейл
Skin	245 057	11	4,00	36,36	Сегментация

Транз.: Количество транзакций, *Плотн.:* Плотность $\left(\frac{\text{Ср. длина}}{\text{Элементы}} \times 100\right)$.

3.2. Эксперименты

Для оценки производительности было спроектировано четыре эксперимента.

Эксперимент 1 исследует зависимость времени выполнения и потребления памяти от ключевых параметров запуска на датасете kosarak: от *minsup* для MaxFEM и AFEM, от *k* для ТКЕ. Три значения *winlen* позволяют проследить, как меняется разрыв между SPMF и Desbordante при различных режимах нагрузки.

Эксперимент 2 охватывает все 12 датасетов и все три алгоритма. Для каждого сочетания датасета, алгоритма и числа потоков вычисляется среднее геометрическое коэффициента ускорения и коэффициента экономии памяти относительно SPMF.

Эксперимент 3 на основе данных эксперимента 2 определяет наибольшее достигнутое ускорение по каждому датасету и анализирует, какие характеристики датасета на него влияют.

Эксперимент 4 проводился на облачном стенде и исследует зависимость ускорения и потребления памяти от числа потоков при увеличении пула от 1 до 32.

3.3. Обсуждение результатов

Эксперимент 1. На рисунке 2 показаны результаты для всех трёх алгоритмов на датасете kosarak.

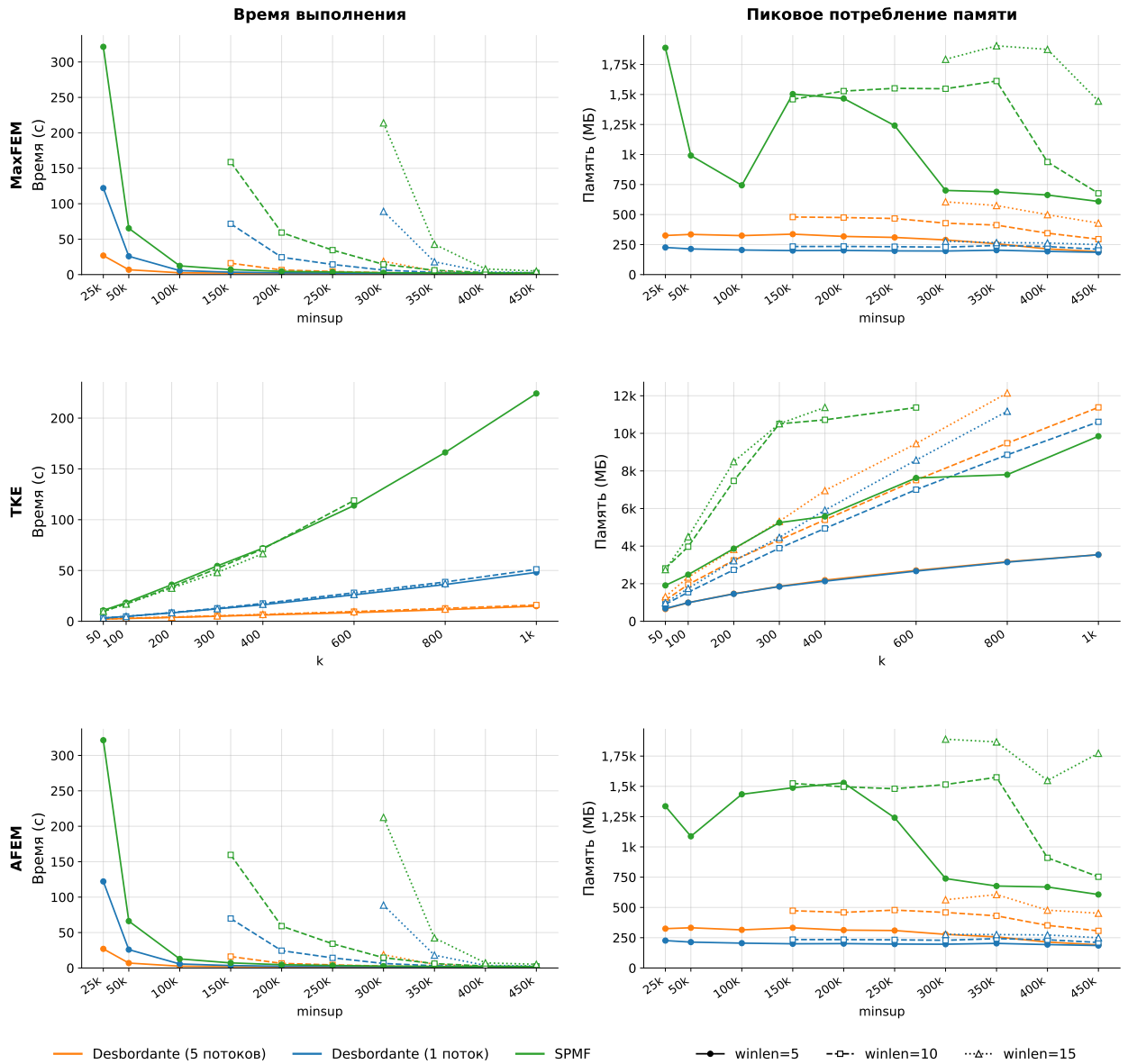


Рис. 2: Сравнение реализаций алгоритмов MaxFEM, TKE и AFEM: время выполнения и пиковое потребление памяти на датасете kosarak с разными входными параметрами.

Для MaxFEM и AFEM кривые времени имеют характерную форму: при высоких значениях *minsup* обе реализации работают быстро, но при снижении порога наступает точка резкого роста, после которой время SPMF возрастает многократно. Положение этой точки сильно зависит от *winlen*: чем больше окно, тем при более высоком *minsup* алгоритм входит в зону тяжёлых вычислений. Потребление памяти у Desbordante при этом остаётся практически постоянным вне зависимости от нагрузки: промежуточные структуры аллоцируются заранее и переиспользуются. SPMF потреб-

ляет в несколько раз больше и демонстрирует выраженные колебания, обусловленные поведением JVM-кучи. Графики MaxFEM и AFEM практически идентичны — их алгоритмическое сходство проявляется и в экспериментальных результатах.

У ТКЕ характер зависимостей иной. Время выполнения растёт с k плавно, без резких скачков; $winlen$ на скорость практически не влияет. Зато память ведёт себя противоположным образом: она заметно растёт с k у обеих реализаций, и $winlen$ здесь оказывает существенное влияние. Отсутствующие точки SPMF при больших k — результат не превышения лимита времени, а нехватки оперативной памяти: при $k=1000$ и $winlen=15$ SPMF ТКЕ завершается с ошибкой OOM.

Эксперимент 2. В таблице 3 приведены средние геометрические коэффициенты ускорения и экономии памяти, усреднённые по всем 12 датасетам.

Таблица 3: Сравнительные результаты производительности алгоритмов.

Датасет	Потоки	MaxFEM		ТКЕ		AFEM	
		Время	Память	Время	Память	Время	Память
accidents	1 поток	6,03	6,61	7,51	6,17	5,80	6,74
	5 потоков	14,77	2,43	13,26	4,73	14,91	2,58
BMS1	1 поток	1,79	13,33	10,74	8,93	2,07	13,36
	5 потоков	4,21	12,77	36,58	8,36	9,59	13,08
BMS2	1 поток	2,06	17,06	4,61	9,24	2,05	17,57
	5 потоков	6,51	16,95	17,79	8,60	6,86	17,40
chainstore	1 поток	1,73	6,01	4,75	3,74	1,73	6,13
	5 потоков	3,43	5,88	14,89	3,75	3,52	5,99
chess	1 поток	4,46	2,67	133,51	10,62	4,75	1,92
	5 потоков	12,49	2,22	254,92	8,89	20,08	1,88
kosarak	1 поток	2,41	6,42	4,14	2,38	2,42	6,29
	5 потоков	9,78	3,32	10,46	2,14	9,80	3,14
mushrooms	1 поток	4,75	2,54	55,58	3,50	4,53	1,73
	5 потоков	21,81	2,18	120,26	3,29	20,61	1,70
OnlineRetail	1 поток	2,93	12,35	7,73	3,13	2,90	11,96
	5 потоков	12,37	10,93	23,65	3,14	12,90	10,63
PAMAP	1 поток	3,70	11,00	10,05	5,17	3,66	11,19
	5 потоков	10,65	6,24	12,59	5,22	10,52	6,25
RecordLink	1 поток	4,40	7,36	12,20	4,63	4,25	7,45
	5 потоков	9,87	3,24	16,89	4,27	9,86	3,38
retail	1 поток	2,18	8,27	3,54	2,59	2,15	6,38
	5 потоков	6,11	4,98	13,24	2,53	9,95	4,45
Skin	1 поток	4,51	15,33	5,22	2,44	4,56	10,36
	5 потоков	21,32	4,41	14,10	2,29	21,29	4,16
В среднем	1 поток	3,14	7,79	9,96	4,56	3,16	6,98
	5 потоков	9,62	4,99	24,11	4,23	11,22	4,79

Примечание: В таблице приведено среднее геометрическое метрик эффективности Desbordante по сравнению с базовой реализацией в SPMF.

Время — коэффициент ускорения ($Time_{SPMF}/Time_{Desb}$). **Память** — коэффициент экономии памяти (Mem_{SPMF}/Mem_{Desb}).

Результаты MaxFEM и AFEM оказались весьма схожими. В однопоточном режиме ускорение составляет в среднем $3,14\times$ для первого алгоритма и $3,16\times$ для второго, при этом экономия памяти достигает $7,79\times$ и $6,98\times$ соответственно. При пяти потоках ускорение достигает $9,62\times$ и $11,22\times$. Небольшое превосходство AFEM в параллельном режиме объясняется тем, что этот алгоритм не требует вычислительных ресурсов на фильтрацию немаксимальных эпизодов при слиянии результатов.

ТКЕ опережает оба алгоритма по ускорению: $9,96\times$ однопоточно и $24,11\times$ при пяти потоках. Объясняется это тем, что реализация ТКЕ в SPMF не включает оптимизацию Temporal Pruning, которая в Desbordante позволяет обрывать построение списков позиций на ранних этапах. Экономия памяти у ТКЕ ниже ($4,56\times$ против $6\text{--}8\times$ у MaxFEM/AFEM).

Эксперимент 3. На рисунке 3 показано максимальное ускорение по каждому датасету.

Выигрыш по времени существенно зависит от характеристик датасета. На плотных датасетах с длинными транзакциями — chess (49%), mushrooms (19%), accidents (7%) — ускорение наиболее высокое: глубокий поиск с большим числом кандидатов сильнее нагружает JVM, тогда как эффективное управление памятью C++ и параллелизация дают здесь максимальную отдачу. На разреженных датасетах с большим словарём событий — chainstore, kosarak, retail (плотность около 0,02–0,06%) — паттернов мало, задача лёгкая, и SPMF справляется быстро; абсолютный разрыв невелик.

Для ТКЕ chess и mushrooms выделяются особо: $133\times$ и $255\times$ для chess, $55\times$ и $120\times$ для mushrooms. Это прямое следствие отсутствия Temporal Pruning в SPMF ТКЕ: на плотных данных без раннего отсека алгоритм вынужден обрабатывать очень большие списки позиций.

Картина по памяти иная: наибольший выигрыш наблюдается не на плотных, а на датасетах с маленьким алфавитом или очень высокой долей уникальных элементов (BMS2: $17\times$, Skin: $15\times$, BMS1: $13\times$). Там вычислительная нагрузка мала, но JVM всё равно резервирует значительный объём под системные структуры — накладные расходы, которых C++ полностью лишён.

Эксперимент 4. На рисунке 4 показаны зависимости ускорения и потребления памяти от числа потоков.

Закон Амдала утверждает, что ускорение при использовании n потоков ограничено последовательной долей программы:

$$S(n) = \frac{1}{(1 - P) + P/n},$$

где $P \in [0, 1]$ — доля кода, допускающая параллельное исполнение. MaxFEM и AFEM

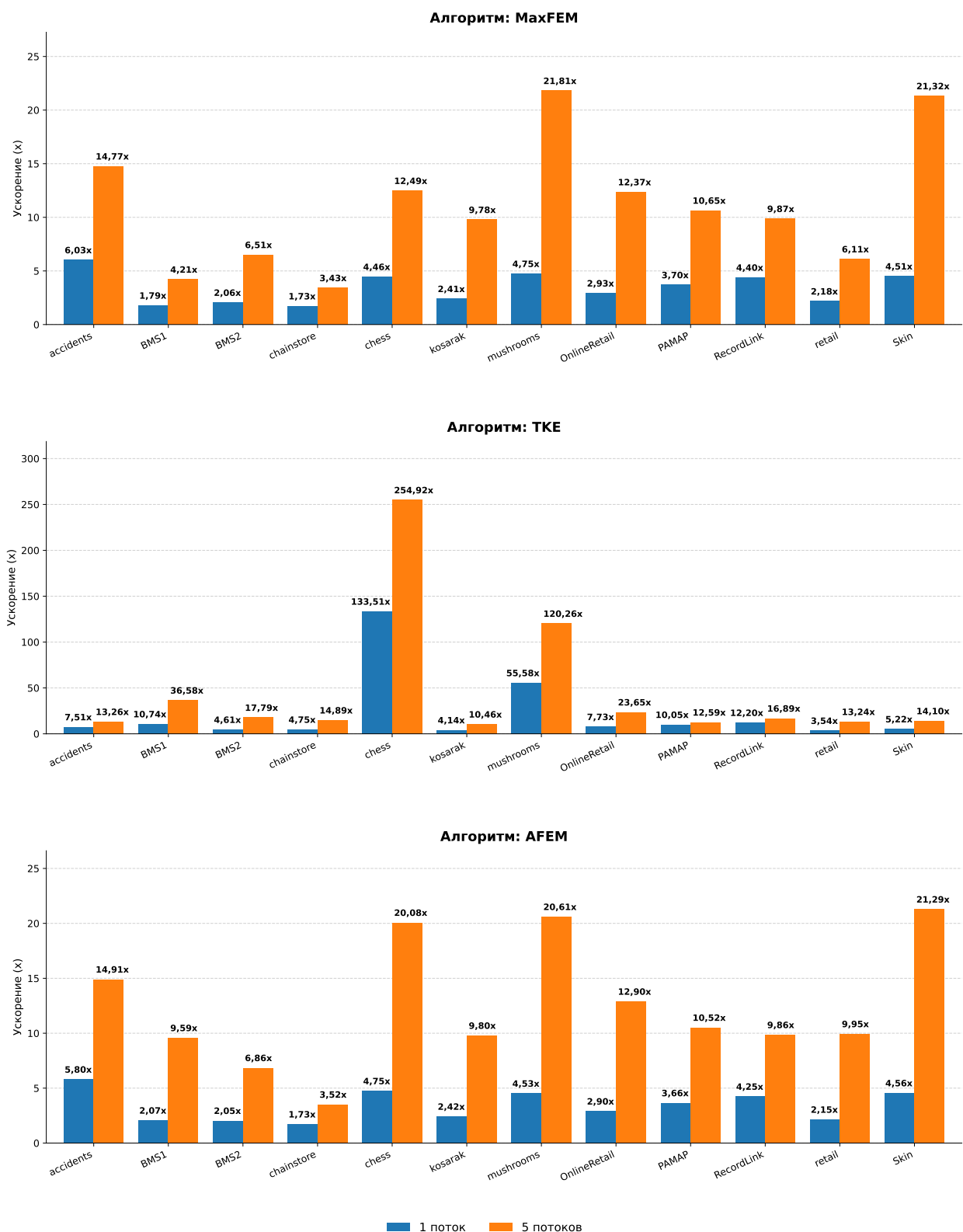


Рис. 3: Максимальное ускорение Desbordante относительно SPMF по датасетам.

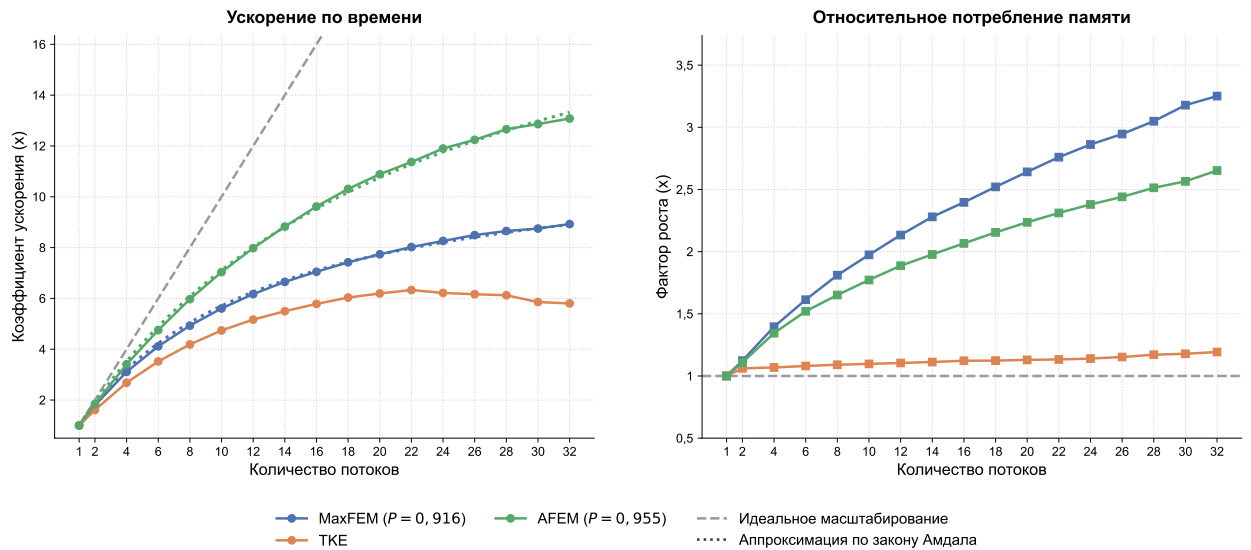


Рис. 4: Масштабируемость параллельных реализаций: коэффициент ускорения (левый граф) и фактор роста потребления памяти (правый граф) в зависимости от числа потоков.

хорошо описываются этим законом. AFEM достигает $\approx 13\times$ при 32 потоках ($P=0,955$); MaxFEM — $\approx 9\times$ ($P=0,916$). Такая разница вызвана необходимостью проверки максимальности эпизодов при слиянии локальных буферов в MaxFEM. Это вносит дополнительный последовательный участок вычислений.

Масштабируемость ТКЕ имеет другой характер. Ускорение растёт до $\approx 6,3\times$ при 22 потоках, а затем начинает снижаться, опускаясь до $\approx 5,8\times$ при 32 потоках. Дегенерация возникает из-за дисбаланса нагрузки между главным и рабочими потоками. При избыточном размере пула рабочие потоки начинают выполнять вычисления быстрее, чем главный поток успевает формировать новые задания и обрабатывать результаты.

В плане потребления памяти наблюдается обратная ситуация. MaxFEM и AFEM при 32 потоках потребляют в 3,3 раза больше, чем в однопоточном режиме. Причина кроется в необходимости аллоцировать независимый буфер под найденные эпизоды внутри каждого отдельного потока. Алгоритм ТКЕ, напротив, крайне экономно расходует память при масштабировании. Рабочим потокам здесь не нужно локально накапливать результаты, из-за чего даже при 32 потоках потребление памяти возрастает лишь в $\approx 1,2$ раза. Таким образом, ТКЕ немного деградирует по времени при большом числе потоков, но выигрывает в памяти при параллельном запуске.

Итог главы. Проведенные эксперименты подтверждают, что реализации алгоритмов в Desbordante стабильно превосходят эталонные версии из SPMF как по скорости, так и по потреблению памяти на всём диапазоне исследованных датасетов и параметров. Перенос с Java на C++ даёт в среднем 3–10-кратный выигрыш по времени в зависимости от алгоритма; параллелизация добавляет ещё около 3-кратного ускорения при пяти потоках. MaxFEM и AFEM масштабируются согласно закону Амдала

и достигают $\approx 9\times$ и $\approx 13\times$ при 32 потоках; ТКЕ при больших пулах потоков демонстрирует деградацию производительности из-за конкуренции за очередь $\text{top-}k$, зато не несёт накладных расходов по памяти. Сравнение ограничено SPMF как единственной публично доступной реализацией всех трёх алгоритмов.

4. Практическое применение

Поиск частотных эпизодов изначально возник из практической задачи анализа журналов аварийных сигналов в телекоммуникационных сетях, где требовалось находить часто повторяющиеся комбинации сигналов [2]. С тех пор спектр приложений существенно расширился. В обзоре Ouaguen и др. [8] к числу практически значимых применений FEM отнесены прогнозирование событий в киберфизических системах (умные дома, автономные автомобили), мониторинг сенсорных сетей на ядерных объектах, оркестрация IoT-систем в промышленном производстве и анализ задержек в железнодорожных сетях. Помимо этого, частотные эпизоды используются для анализа финансовых временных рядов, исследования веб-логов, диагностики причин отказов оборудования, выявления аномалий в сетевом трафике и обнаружения вторжений в компьютерные системы. Объединяющая черта этих задач — наличие длинной последовательности событий, в которой важны не отдельные элементы, а устойчивые подпоследовательности, в том числе содержащие одновременно происходящие события.

В качестве иллюстрации в настоящей главе рассматривается одна из перечисленных задач — обнаружение вторжений в компьютерные системы на основе трасс системных вызовов. Выбор обусловлен тем, что это одна из наиболее изученных областей применения FEM с общедоступным набором данных и устоявшимися протоколами оценки, что делает демонстрацию воспроизводимой и сопоставимой с литературой. Используемый в главе пайплайн — базовый: эпизоды извлекаются один раз и далее используются как признаки стандартного классификатора. В более развитых системах FEM встраивается как один из этапов многослойной аналитики — например, при предсказании событий в киберфизических системах [8] из разнородных источников сначала извлекаются типизированные события, по ним восстанавливаются серийные эпизоды, образующие граф переходов, и уже на этом графе строится предсказательная модель. Цель настоящей главы — показать, что реализованные алгоритмы пригодны для построения такого рода пайплайнов на реальных данных, а получаемые ими эпизоды несут содержательный сигнал о происходящем в системе.

4.1. Набор данных ADFA-LD

ADFA-LD — общедоступный набор данных, разработанный в Австралийской академии обороны для оценки методов обнаружения вторжений на уровне хоста [7]. Он содержит трассы системных вызовов, собранные на современной Linux-системе с актуальным на момент создания набором уязвимостей и атак, и применяется в исследованиях систем обнаружения вторжений уровня хоста (host-based IDS) как стандартный эталон.

Каждая трасса — это последовательность целочисленных идентификаторов си-

стемных вызовов, сделанных одним процессом за время его жизни. Набор разделён на три части:

- **Training_Data_Master** — 833 трассы нормального поведения, полученные при штатной работе системы;
- **Validation_Data_Master** — 4372 дополнительные нормальные трассы;
- **Attack_Data_Master** — атакующие трассы, сгруппированные в шесть категорий: **Adduser**, **Hydra_FTP**, **Hydra_SSH**, **Java_Meterpreter**, **Meterpreter** и **Web_Shell**.

Атакующих трасс заметно меньше нормальных, что отражает реальное распределение событий в production-системах и делает задачу несбалансированной. Для эксперимента нормальные и атакующие трассы были стратифицированно разделены на обучающую, валидационную и тестовую выборки. В обучающую выборку вошли все 833 базовые нормальные трассы и половина атакующих по каждой категории; валидационная и тестовая части набирались из оставшихся данных. Тестовая выборка использовалась только для финальной оценки, после фиксации всех параметров на валидации.

Каждая трасса в ADFA-LD представляет собой одну строку — последовательность целых чисел, разделённых пробелами. Это не совпадает с принятым в Desbordante многопоследовательным форматом, в котором событийная последовательность подаётся как поток наборов событий с временными метками. Поэтому перед запуском алгоритмов трассы были приведены к единому формату: каждая трасса записывалась как непрерывный участок событий с возрастающими метками, а между трассами вставлялся разрыв длины *winlen*, превышающий ширину окна. Это гарантирует, что найденные эпизоды не могут содержать вхождений, охватывающих сразу две различные трассы.

4.2. Признаковое пространство на основе частотных эпизодов

Трассы системных вызовов обладают двумя характерными свойствами. Во-первых, отдельные вызовы малоинформативны: вредоносное поведение проявляется не в самих вызовах, а в их повторяющихся комбинациях. Во-вторых, такие комбинации устойчивы к локальным вариациям — одна и та же логика может реализовываться через несколько различных, но порождающих похожие подпоследовательности путей. Это делает событийные эпизоды естественным языком описания: композитный эпизод фиксирует упорядоченные группы вызовов, а ширина окна задаёт допустимую вариативность между ними.

Признаковое пространство строилось в два этапа.

Этап 1. Глобальный словарь эпизодов. На обучающей части отдельно для нормальных и атакующих трасс запускался алгоритм ТКЕ с параметрами $k = 1000$ и $winlen = 4$. Раздельный запуск обусловлен сильным дисбалансом классов: при едином запуске top- k эпизодов был бы почти целиком сформирован нормальными трассами, и специфические для атак паттерны не попали бы в словарь. После объединения двух множеств с удалением дубликатов получено 1617 уникальных эпизодов, которые далее используются как признаки. Примеры наиболее частотных эпизодов в каждом классе приведены в таблице 4: в нормальных трассах доминируют вызовы 3 и 4 (`read`, `write`), в атакующих — 168 и 265 (`poll`, `getuid32`).

Таблица 4: Топ-5 наиболее частотных эпизодов в каждом классе

Нормальные		Атакующие	
Эпизод	Поддержка	Эпизод	Поддержка
$\langle\{3\}\rangle$	55 664	$\langle\{168\}\rangle$	37 773
$\langle\{3\}, \{3\}\rangle$	39 478	$\langle\{265\}\rangle$	30 466
$\langle\{4\}\rangle$	31 342	$\langle\{168\}, \{168\}\rangle$	29 875
$\langle\{3\}, \{3\}, \{3\}\rangle$	28 038	$\langle\{168\}, \{265\}\rangle$	23 434
$\langle\{4\}, \{4\}\rangle$	27 132	$\langle\{3\}\rangle$	22 783

Этап 2. Векторизация трасс. Для каждой трассы — как обучающей, так и валидационной/тестовой — запускался ТКЕ с теми же параметрами. Полученные значения поддержки сопоставлялись с глобальным словарём: если эпизод входил и в глобальный словарь, и в локальный top- k трассы, то соответствующий признак получал значение его локальной поддержки, иначе ноль. Такая схема эквивалентна неявной регуляризации — в трассе фиксируются лишь её доминирующие паттерны, шумовые редкие подпоследовательности отбрасываются.

Полученная матрица признаков размера 2247×1617 далее нормировалась по длине трассы и взвешивалась по схеме TF-IDF, что снижает влияние эпизодов, встречающихся почти во всех трассах. Статистики IDF подбирались только по обучающей части во избежание утечки информации в валидационную и тестовую выборки.

Классификация. Полученные векторы признаков подавались в два классических классификатора: случайный лес и SVM с RBF-ядром. Оба классификатора обучались с балансировкой весов классов, чтобы скомпенсировать дисбаланс выборки.

4.3. Результаты и сравнение с n -граммной моделью

В качестве базовой модели использовался стандартный подход на основе n -грамм: для каждой трассы строилось TF-IDF-представление в пространстве 1–3-грамм си-

стемных вызовов с ограничением словаря 2000 признаками. Такой подход не использует понятия эпизода и временного окна — он рассматривает трассу как обычную последовательность токенов.

Используются значения Recall при FPR=5 %, Recall при FPR=10 % и AUC; 95 % доверительные интервалы получены методом bootstrap по трассам тестового множества. Результаты приведены в таблице 5.

Таблица 5: Качество детектора на тестовой выборке ADFA-LD

Модель	Recall@FPR=5 %	Recall@FPR=10 %	AUC
FEM + RF	0,592 [0,484–0,682]	0,886 [0,830–0,939]	0,944 [0,933–0,954]
FEM + SVM, $C=1$	0,752 [0,675–0,840]	0,937 [0,898–0,966]	0,954 [0,942–0,964]
FEM + SVM, $C=10$	0,646 [0,500–0,752]	0,932 [0,893–0,966]	0,953 [0,944–0,962]
n -gram + RF	0,559 [0,463–0,626]	0,906 [0,815–0,947]	0,944 [0,933–0,954]
n -gram + SVM, $C=1$	0,578 [0,519–0,680]	0,898 [0,854–0,947]	0,950 [0,939–0,959]
n -gram + SVM, $C=10$	0,539 [0,432–0,655]	0,903 [0,859–0,951]	0,945 [0,935–0,955]
FEM + n -gram, SVM $C=1$	0,694 [0,549–0,811]	0,951 [0,917–0,981]	0,954 [0,945–0,962]

Примечание. В скобках — 95 % доверительные интервалы.

FEM-модели по AUC и полноте в рассмотренных рабочих точках выходят на уровень n -граммного представления: интервалы для AUC и для Recall@FPR=10 % существенно пересекаются. В области FPR=5 % точечные оценки полноты у FEM-моделей выше, чем у n -граммных, но из-за небольшого числа атак на тесте доверительные интервалы остаются широкими, и устойчиво говорить о превосходстве в этой точке не приходится. При этом FEM-представление достигает сопоставимого качества при меньшем числе признаков (1617 против 2000) и без явного перебора n -грамм по разным значениям n — ширина окна $winlen = 4$ единым образом охватывает паттерны разной длины и разной плотности. По стоимости извлечения признаков FEM-модель ожидаемо дороже n -граммной — поиск эпизодов и подсчёт их поддержки занимают около 30 секунд против одной секунды у n -граммного векторизатора, — но в абсолютных значениях эта стоимость невелика и допустима для офлайн-анализа на данных подобного объёма.

Поведение на отдельных категориях атак. Усреднённые метрики не позволяют судить о том, какие именно классы атак уверенно распознаются. В таблице 6 приведена полнота FEM-модели на валидационной выборке отдельно по каждой категории.

Видно, что брутфорс-атаки на сетевые сервисы (Hydra_FTP, Hydra_SSH) распознаются практически безошибочно: они порождают характерные циклические последовательности, которые ТКЕ надёжно отлавливает уже на коротких окнах. Наоборот, категория Adduser распознаётся существенно хуже: соответствующие трассы кратки, их немного, и поведение злоумышленника по своему характеру близко к легитимным

Таблица 6: Полнота на валидации по категориям атак (FEM + SVM, $C=1$)

Категория	Recall
Hydra_FTP	97,4 %
Hydra_SSH	95,5 %
Java_Meterpreter	93,5 %
Web_Shell	89,3 %
Meterpreter	88,2 %
Adduser	77,3 %

административным действиям, что и проявляется в относительно низкой полноте.

Сопоставление с публикациями по ADFA-LD. Полученные числа полезно поместить в контекст известных результатов на ADFA-LD.

Creesh и Hu [25] применяют машину экстремального обучения (ELM) с семантическим представлением системных вызовов и достигают $AUC \approx 0,954$, полноты около 0,85 при $FPR=10\%$ и около 0,73 при $FPR=5\%$.

Khreich и др. [26] используют n -граммы переменной длины (до длины 6) в сочетании с одноклассовым SVM и сообщают $AUC \approx 0,955$, полноту около 0,93 при $FPR=10\%$ и около 0,8 при $FPR=5\%$.

Из более современных результатов: Liao и др. [27] комбинируют признаки фиксированной и переменной длины с релаксированной SVM и достигают $AUC \approx 0,99$ при уровне ложных срабатываний около 2,4 %.

Обсуждение. Результаты подтверждают основную цель эксперимента: реализованные алгоритмы успешно интегрируются в аналитические пайплайны для обработки данных, не уступая по качеству традиционным подходам. Эпизоды дают компактное и интерпретируемое представление последовательностей. На категориях атак с выраженным повторяющимся поведением (Hydra_FTP, Hydra_SSH, Java_Meterpreter) модель генерирует наиболее качественные результаты. Это показывает, что $top-k$ эпизоды действительно захватывают характерные для атак циклические конструкции.

Дальнейшие направления развития могут быть связаны с расширением признакового представления. Например, сочетание эпизодных признаков с другими типами представлений. Также можно рассмотреть переход от частотных эпизодов к эпизодным правилам (episode rules) [8]. Они фиксируют имплицативные связи вида “эпизод α обычно влечёт эпизод β ”. Такие правила более тесно соотносятся с конкретными сценариями поведения и пригодны не только для классификации, но и для интерпретации обнаруженных аномалий.

Итог главы. На примере одной из распространённых областей применения FEM — обнаружения вторжений на основе трасс системных вызовов — продемонстрировано,

что реализованные в работе алгоритмы можно использовать при построении аналитического пайплайна на реальных данных. Признаковое пространство, построенное на частотных эпизодах, сопоставимо с устоявшимися n -граммными и другими классическими моделями для ADFA-LD. Более того, извлечённые эпизоды содержательно интерпретируемы и непосредственно отражают характерные для атак цепочки системных вызовов.

Заключение

В рамках настоящей работы в систему профилирования данных Desbordante введена поддержка анализа событийных последовательностей. Реализованы три алгоритма поиска частотных эпизодов: MaxFEM, выделяющий лишь максимальные шаблоны; ТКЕ, в котором пользователь задаёт желаемое число шаблонов k вместо порога поддержки; и AFEM, решающий исходную задачу FEM в её классической постановке. До этого момента в Desbordante отсутствовали инструменты для работы с данными подобного типа, поэтому помимо самих алгоритмов был спроектирован и добавлен общий слой представления событийных последовательностей и интерфейс их подачи на вход профилировщику.

Все три алгоритма перенесены с языка Java на C++ с применением системных оптимизаций структур данных и шаблонов доступа к памяти. Для каждого из них дополнительно построена параллельная версия. Реализации интегрированы в ядро Desbordante и доступны через нативный Python-интерфейс системы, что позволяет использовать их в составе сторонних аналитических пайплайнов.

Эффективность реализаций экспериментально оценена в сравнении с эталонными версиями из библиотеки SPMF на двенадцати общедоступных наборах данных из различных предметных областей. В однопоточном режиме перенос с Java на C++ обеспечивает среднее геометрическое ускорение в $3,14\times$ для MaxFEM, $3,16\times$ для AFEM и $9,96\times$ для ТКЕ при экономии оперативной памяти в $7,79\times$, $6,98\times$ и $4,56\times$ соответственно. Параллельная версия на пяти ядрах увеличивает ускорение до $9,62\times$ у MaxFEM, $11,22\times$ у AFEM и $24,11\times$ у ТКЕ. Дополнительный анализ масштабируемости на стенде с 32 ядрами показал, что MaxFEM и AFEM описываются законом Амдала; в пределе ускорение, достижимое за счёт параллелизации, оценивается в среднем в $\approx 11\times$ для MaxFEM и $\approx 22\times$ для AFEM относительно однопоточной реализации. ТКЕ упирается в конкуренцию потоков за очередь $\text{top-}k$ и масштабируется до $\approx 6,3\times$ при 22 потоках. Стабильное превосходство по обоим метрикам наблюдается на всём диапазоне исследованных параметров.

Прикладная применимость реализованных алгоритмов продемонстрирована на задаче обнаружения вторжений на основе трасс системных вызовов с использованием набора данных ADFA-LD. Построенный на $\text{top-}k$ эпизодах детектор показал качество, сопоставимое со стандартным n -граммным подходом, при меньшей размерности признакового пространства. Извлечённые эпизоды содержательно интерпретируемы и непосредственно соотносятся с характерными для атак цепочками вызовов, что подтверждает ценность реализованных алгоритмов как практического инструмента анализа событийных последовательностей.

Часть результатов работы — реализация и параллелизация алгоритма MaxFEM — прошла внешнюю рецензию и опубликована в материалах 39-й международной кон-

ференции FRUCT [28].

Естественным продолжением работы является расширение спектра поддерживаемых задач в том же семействе. Добавленный в Desbordante слой событийных последовательностей не ограничен тремя реализованными алгоритмами и может служить основой для других задач того же семейства — например, поиска замкнутых эпизодов или эпизодных правил, а также для работы с альтернативными мерами частотности [8].

Список литературы

- [1] Data Profiling / Abedjan Ziawasch, Golab Lukasz, Naumann Felix, and Papenbrock Thorsten. — Morgan & Claypool Publishers, 2018. — ISBN: [1681734486](#).
- [2] Mannila Heikki, Toivonen Hannu, Inkeri Verkamo A. Discovery of Frequent Episodes in Event Sequences // [Data Min. Knowl. Discov.](#) — 1997. — Jan. — Vol. 1, no. 3. — P. 259–289. — Access mode: <https://doi.org/10.1023/A:1009748302351>.
- [3] MaxFEM: Mining Maximal Frequent Episodes in Complex Event Sequences / Fournier-Viger Philippe, Nawaz M. Saqib, He Yulin, Wu Youxi, Nouioua Farid, and Yun Unil // *Multi-disciplinary Trends in Artificial Intelligence* / ed. by Surinta Olarik, Kam Fung Yuen Kevin. — Cham : Springer International Publishing. — 2022. — P. 86–98.
- [4] TKE: Mining Top-K Frequent Episodes / Fournier-Viger Philippe, Yang Yanjun, Yang Peng, Lin Jerry Chun-Wei, and Yun Unil // *Trends in Artificial Intelligence Theory and Applications. Artificial Intelligence Practices* / ed. by Fujita Hamido, Fournier-Viger Philippe, Ali Moonis, Sasaki Jun. — Cham : Springer International Publishing. — 2020. — P. 832–845.
- [5] The SPMF Open-Source Data Mining Library Version 2 / Fournier-Viger Philippe, Lin Jerry Chun-Wei, Gomariz Antonio, Gueniche Ted, Soltani Azadeh, Deng Zhihong, and Lam Hoang Thanh // *Machine Learning and Knowledge Discovery in Databases* / ed. by Berendt Bettina, Bringmann Björn, Fromont Élisabeth et al. — Cham : Springer International Publishing. — 2016. — P. 36–40.
- [6] [Desbordante: from benchmarking suite to high-performance science-intensive data profiler](#) / Chernishev George, Polyntsov Michael, Chizhov Anton, Stupakov Kirill, Shchuckin Ilya, Smirnov Alexander, Strutovsky Maxim, Shlyonskikh Alexey, Firsov Mikhail, Manannikov Stepan, Bobrov Nikita, Goncharov Daniil, Barutkin Ilia, Yakshigulov Vadim, Shalnev Vladislav, Muraviev Kirill, Rakhmukova Anna, Shcheka Dmitriy, Chernikov Anton, Kuzin Yakov, Sinelnikov Michael, Abrosimov Grigorii, Popov Dmitriy, Demchenko Artem, Belokonny Sergey, Soloveva Liana-Iuliia, Kurbatov Yaroslav, Vyrolov Mikhail, Saliou Arthur, Gaisin Eduard, and Smirnov Kirill // *Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD)*. — New York, NY, USA : Association for Computing Machinery. — 2025. — CODS-COMAD '24. — P. 234–243. — Access mode: <https://doi.org/10.1145/3703323.3703725>.

- [7] Creech Gideon, Hu Jiankun. Generation of a new IDS test dataset: Time to retire the KDD collection. — 2013. — P. 4487–4492.
- [8] Ouarem Oualid, Nouioua Farid, Fournier-Viger Philippe. A survey of episode mining // [WIREs Data Mining and Knowledge Discovery](#). — 2024. — Vol. 14, no. 2. — P. e1524. — <https://wires.onlinelibrary.wiley.com/doi/pdf/10.1002/widm.1524>.
- [9] Casas-Garriga Gemma. Discovering Unbounded Episodes in Sequential Data // *Knowledge Discovery in Databases: PKDD 2003* / ed. by Lavrač Nada, Gamberger Dragan, Todorovski Ljupčo, Blockeel Hendrik. — Berlin, Heidelberg : Springer Berlin Heidelberg. — 2003. — P. 83–94.
- [10] [Discovering and learning sensational episodes of news events](#) / Ao Xiang, Luo Ping, Li Chengkai, Zhuang Fuzhen, He Qing, and Shi Zhongzhi // *Proceedings of the 23rd International Conference on World Wide Web*. — New York, NY, USA : Association for Computing Machinery. — 2014. — WWW '14 Companion. — P. 217–218. — Access mode: <https://doi.org/10.1145/2567948.2577290>.
- [11] Huang Kuo-Yu, Chang Chia-Hui. Efficient mining of frequent episodes from complex sequences // *Inf. Syst.* — 2008. — Mar. — Vol. 33, no. 1. — P. 96–114. — Access mode: <https://doi.org/10.1016/j.is.2007.07.003>.
- [12] Ouarem Oualid, Nouioua Farid, Fournier-Viger Philippe. [Mining Episode Rules from Event Sequences Under Non-overlapping Frequency](#) // *Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices: 34th International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems, IEA/AIE 2021, Kuala Lumpur, Malaysia, July 26–29, 2021, Proceedings, Part I*. — Berlin, Heidelberg : Springer-Verlag. — 2021. — P. 73–85. — Access mode: https://doi.org/10.1007/978-3-030-79457-6_7.
- [13] [Mining Partially-Ordered Episode Rules in an Event Sequence](#) / Fournier-Viger Philippe, Chen Yangming, Nouioua Farid, and Lin Jerry Chun-Wei // *Intelligent Information and Database Systems: 13th Asian Conference, ACIIDS 2021, Phuket, Thailand, April 7–10, 2021, Proceedings*. — Berlin, Heidelberg : Springer-Verlag. — 2021. — P. 3–15. — Access mode: https://doi.org/10.1007/978-3-030-73280-6_1.
- [14] [A General Depth-First-Search based Algorithm for Frequent Episode Discovery](#) / Zhu Huisheng, Chen Lin, Li Jinhai, Zhou Aiping, Wang Peng, and Wang Wei // *2018 14th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*. — 2018. — P. 890–899.

- [15] [Efficient Episode Mining with Minimal and Non-overlapping Occurrences](#) / Zhu Huisheng, Wang Peng, He Xianmang, Li Yujia, Wang Wei, and Shi Baile // 2010 IEEE International Conference on Data Mining. — 2010. — P. 1211–1216.
- [16] [Discovering Frequent Closed Episodes from an event sequence](#) / Zhu Huisheng, Wang Peng, Wang Wei, and Shi Baile // The 2012 International Joint Conference on Neural Networks (IJCNN). — 2012. — P. 1–8.
- [17] Zhou Wenzhi, Liu Hongyan, Cheng Hong. [Mining closed episodes from event sequences efficiently](#) // Proceedings of the 14th Pacific-Asia Conference on Advances in Knowledge Discovery and Data Mining - Volume Part I. — Berlin, Heidelberg : Springer-Verlag. — 2010. — PAKDD'10. — P. 310–318. — Access mode: https://doi.org/10.1007/978-3-642-13657-3_34.
- [18] Tatti Nikolaj, Cule Boris. Mining closed strict episodes // [Data Mining and Knowledge Discovery](#). — 2011. — Vol. 25, no. 1. — P. 34–66. — Access mode: <http://dx.doi.org/10.1007/s10618-011-0232-z>.
- [19] Achar Avinash, Abdul Majeed Ibrahim, Sastry P.S. Pattern-growth based frequent serial episode discovery // [Data & Knowledge Engineering](#). — 2013. — 09. — Vol. 87. — P. 91–108.
- [20] Méger Nicolas, Rigotti Christophe. Constraint-based mining of episode rules and optimal window sizes // Proceedings of the 8th European Conference on Principles and Practice of Knowledge Discovery in Databases. — Berlin, Heidelberg : Springer-Verlag. — 2004. — PKDD '04. — P. 313–324.
- [21] Ma Xi, Pang HweeHwa, Tan Kian-Lee. Finding Constrained Frequent Episodes Using Minimal Occurrences // Proceedings of the Fourth IEEE International Conference on Data Mining. — USA : IEEE Computer Society. — 2004. — ICDM '04. — P. 471–474.
- [22] Laxman Srivatsan, Sastry P. S., Unnikrishnan K. P. [A fast algorithm for finding frequent episodes in event streams](#) // Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. — New York, NY, USA : Association for Computing Machinery. — 2007. — KDD '07. — P. 410–419. — Access mode: <https://doi.org/10.1145/1281192.1281238>.
- [23] Online Frequent Episode Mining / Ao Xiang, Luo Ping, Li Chengkai, Zhuang Fuzhen, and He Qing // [Proceedings - International Conference on Data Engineering](#). — 2015. — 05. — Vol. 2015. — P. 891–902.
- [24] Counting the frequency of time-constrained serial episodes in a streaming sequence / Li Hui, Peng Sizhe, Li Jian, Li Jingjing, Cui Jiangtao, and Ma Jianfeng // [Inf. Sci.](#) —

2019. — Dec. — Vol. 505, no. C. — P. 422–439. — Access mode: <https://doi.org/10.1016/j.ins.2019.07.098>.
- [25] Creech Gideon, Hu Jiankun. A Semantic Approach to Host-Based Intrusion Detection Systems Using Contiguous and Discontiguous System Call Patterns // [IEEE Transactions on Computers](#). — 2014. — Vol. 63, no. 4. — P. 807–819.
- [26] An anomaly detection system based on variable N-gram features and one-class SVM / Khreich Wael, Khosravifar Babak, Hamou-Lhadj Abdelwahab, and Talhi Chamseddine // *Inf. Softw. Technol.* — 2017. — Nov. — Vol. 91, no. C. — P. 186–197.
- [27] Liao Xiaoyao, Wang Changzhi, Chen Wen. Anomaly Detection of System Call Sequence Based on Dynamic Features and Relaxed-SVM // [Security and Communication Networks](#). — 2022. — Vol. 2022, no. 1. — P. 6401316.
- [28] [Scalable Maximal Frequent Episode Mining with Desbordante](#) / Ivanov Maxim, Smirnov Matvei, Strazdina Alisa, and Chernishev George // 2026 39th Conference of Open Innovations Association (FRUCT). — 2026. — P. 102–113.